Magpie — Sprint 2 Technical Spec (Phase 2: Research & Valuation)

Audience: implementing agent (Codex) + Regan. **Builds on** Phase 1 (`SPRINT_0_1_SPEC.md`).
Status: planning / review — implement only after this spec is reviewed and approved.

> Build only what is in this document (the locked Sprint 2 scope, D-23…D-30). Everything beyond it — live metals pricing, eBay, OCR, AI, listing generation, realized-profit dashboards, full category schemas — is a later sprint with a seam, not a build target.

Conventions inherited from `SPRINT_0_1_SPEC.md` (not restated here): the `TimeStampedUUIDModel` base (UUID PK + timestamps); modular monolith under `apps/` ; adapter ports under `integrations/` ; DRF defaults (PageNumberPagination 50, `SearchFilter` + `DjangoFilterBackend` + `OrderingFilter` , `SessionAuthentication` , `IsAuthenticated`); the Vite dev proxy; settings split; and `manage.py backup` must include the new tables. UUID PKs and the per-category JSONB `attributes` field carry over unchanged.

---

0. Locked decisions this spec implements

- **D-23** Sprint 2 = first **zero-API** research & valuation slice.
- **D-24** One `Comparable` model, `kind` discriminator; sold comps human-fed; no ingestion / scraping / warehousing.
- **D-25** Thin `ResearchRecord` .
- **D-26** `FeeCalculator` + **data-driven, dated** `FeeSchedule` ; no hardcoded rates.
- **D-27** `ValuationReport` = source of record; `is_current` ; item fields denormalized + synced; first-class override-with-reason; confidence + provenance.
- **D-28** Strategy seam: `comp_based` + `commodity_manual` (Sprint 2), `commodity_live` deferred.
- **D-29** Assisted research = URL-template helpers only.
- **D-30** Out-of-scope list (see §10).

---

1. New apps & repo additions

```
backend/apps/
├── research/                # NEW
│   ├── models.py            # Comparable, ResearchRecord
│   ├── links.py             # assisted-research URL templates (pure functions)
│   ├── serializers.py
│   ├── filters.py
│   ├── views.py
│   └── tests/
└── valuation/               # NEW
    ├── models.py            # FeeSchedule, ValuationReport, ValuationComparable (thro
    ├── strategies.py        # ValuationStrategy protocol + comp_based / commodity_man
    ├── services.py          # FeeCalculator (pure), report build/sync helpers
    ├── serializers.py
    ├── filters.py
    ├── views.py
    └── tests/
```

- `integrations/metals.py` **remains a STUB —** `MetalsPriceAdapter.spot_price(...)` **keeps raising** `NotImplementedError` **. Live pricing is Sprint 3.**
- `INSTALLED_APPS += "apps.research", "apps.valuation"` **.**
- `inventory` **gets a small additive migration (new cost fields, §3.6).**
- **No new third-party dependencies.**

---

2. App responsibilities

App	Owns	Depends on
`research`	`Comparable` , `ResearchRecord` , assisted-research link helpers	`inventory`
`valuation`	`FeeSchedule` , `FeeCalculator` , `ValuationReport` , `ValuationComparable` , strategies	`inventory` , `research`

Dependency direction is one-way: `valuation` → `research` → `inventory` **. Nothing in** `inventory` **imports the new apps.**

---

3. Models

3.1 `research.Comparable`

```
class Comparable(TimeStampedUUIDModel):
    class Kind(models.TextChoices):
        ACTIVE = "active", "Active listing"
        SOLD = "sold", "Sold / completed"
        DEALER = "dealer", "Dealer asking"
        CATALOGUE = "catalogue", "Catalogue value"
        MANUAL_ESTIMATE = "manual_estimate", "Manual estimate"
        AUCTION_RESULT = "auction_result", "Auction result"

    item = models.ForeignKey("inventory.InventoryItem", on_delete=models.CASCADE, relat
    kind = models.CharField(max_length=20, choices=Kind.choices)
    source = models.CharField(max_length=200, blank=True, default="")   # "eBay AU (Ter
    title = models.CharField(max_length=300, blank=True, default="")    # the comp's ow
    price = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank=True)
    shipping = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank=Tr
    currency = models.CharField(max_length=3, default="AUD")
    condition = models.CharField(max_length=20, blank=True, default="")  # free text or
    url = models.URLField(blank=True, default="")
    observed_on = models.DateField(null=True, blank=True)               # when the pric
    notes = models.TextField(blank=True, default="")                    # provenance /

    class Meta:
        ordering = ["-observed_on", "-created_at"]
        indexes = [models.Index(fields=["item"]), models.Index(fields=["kind"])]
```

A `Comparable` is **Regan's own captured snapshot** (e.g. a sold price he read in Terapeak), with source + date. There is **no API ingestion and no scraping** — every row is entered by hand or via the UI form (F-09, D-24).

3.2 `research.ResearchRecord` (thin)

```
class ResearchRecord(TimeStampedUUIDModel):
    item = models.ForeignKey("inventory.InventoryItem", on_delete=models.CASCADE, relat
    source = models.CharField(max_length=200, blank=True, default="")   # where / what
    content = models.TextField(blank=True, default="")                  # the note
    links = models.JSONField(default=list, blank=True)                  # [{ "label": s

    class Meta:
        ordering = ["-created_at"]
```

No status/workflow engine — it is a dated log of what was checked and where (D-25).

3.3 `valuation.FeeSchedule` (data-driven, dated)

```
class FeeSchedule(TimeStampedUUIDModel):
    name = models.CharField(max_length=120)                 # "eBay AU 2026 (store)"
    effective_from = models.DateField()
```

```
    is_active = models.BooleanField(default=True)
    final_value_pct = models.DecimalField(max_digits=6, decimal_places=3)        # % of
    per_order_fee = models.DecimalField(max_digits=8, decimal_places=2, default=0)
    promoted_pct = models.DecimalField(max_digits=6, decimal_places=3, default=0)
    gst_pct = models.DecimalField(max_digits=6, decimal_places=3, default=0)     # AU G
    default_packaging_cost = models.DecimalField(max_digits=8, decimal_places=2, defaul
    default_outbound_shipping = models.DecimalField(max_digits=8, decimal_places=2, def
    notes = models.TextField(blank=True, default="")

    class Meta:
        ordering = ["-effective_from"]
```

> **No fee percentage is hardcoded anywhere in the codebase.** Every rate is a row Regan edits (Django admin in Sprint 2 — no dedicated UI yet). The exact current eBay AU rates are a **data-entry task at population time**, not code. Dated so historical reports can pin the schedule that was in effect (§3.4).

3.4 `valuation.ValuationReport` (source of record)

```
class ValuationReport(TimeStampedUUIDModel):
    class Strategy(models.TextChoices):
        COMP_BASED = "comp_based", "Comparable-based"
        COMMODITY_MANUAL = "commodity_manual", "Commodity (manual inputs)"
        COMMODITY_LIVE = "commodity_live", "Commodity (live spot) — Sprint 3"

    item = models.ForeignKey("inventory.InventoryItem", on_delete=models.CASCADE, relat
    strategy = models.CharField(max_length=20, choices=Strategy.choices, default=Strate
    is_current = models.BooleanField(default=False)

    # computed estimate (over the INCLUDED comps, or commodity inputs)
    estimate_low = models.DecimalField(max_digits=12, decimal_places=2, null=True, blan
    estimate_median = models.DecimalField(max_digits=12, decimal_places=2, null=True, b
    estimate_high = models.DecimalField(max_digits=12, decimal_places=2, null=True, bla

    # price points (D-26/D-27)
    suggested_price = models.DecimalField(max_digits=12, decimal_places=2, null=True, b
    fast_sale_price = models.DecimalField(max_digits=12, decimal_places=2, null=True, b
    patient_price = models.DecimalField(max_digits=12, decimal_places=2, null=True, bla
    min_acceptable_price = models.DecimalField(max_digits=12, decimal_places=2, null=Tr
    currency = models.CharField(max_length=3, default="AUD")

    # confidence + provenance
    confidence_score = models.FloatField(null=True, blank=True)     # 0-1, human-set (o
    confidence_reason = models.TextField(blank=True, default="")

    # first-class manual override
    is_overridden = models.BooleanField(default=False)
    override_reason = models.TextField(blank=True, default="")      # required by seria
```

```
    # strategy inputs (e.g. commodity_manual: {metal, weight_g, fineness, spot_price_pe
    inputs = models.JSONField(default=dict, blank=True)

    # fee basis pinned for reproducible profit math (the schedule used at creation)
    fee_schedule = models.ForeignKey("valuation.FeeSchedule", null=True, blank=True, on
                                     related_name="valuation_reports")
    notes = models.TextField(blank=True, default="")

    comparables = models.ManyToManyField("research.Comparable", through="valuation.Valu
                                         related_name="valuation_reports")

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.UniqueConstraint(fields=["item"], condition=models.Q(is_current=True
                                    name="one_current_valuation_per_item"),
        ]
```

`set_current()` **(in** `valuation/services.py` **), inside a transaction: clear any other** `is_current` **report for the item, set this one, and sync the item's denormalized fields**: `item.estimated_value ← estimate_median` , `item.target_price ← suggested_price` , `item.min_price ← min_acceptable_price` **(only overwriting non-null values).** `fast_sale_price` **/** `patient_price` **live on the report and drive the profit projections (§3.8) without a dedicated item column.**

3.5 `valuation.ValuationComparable` (through-model — the provenance link)

```
class ValuationComparable(TimeStampedUUIDModel):
    report = models.ForeignKey("valuation.ValuationReport", on_delete=models.CASCADE, r
    comparable = models.ForeignKey("research.Comparable", on_delete=models.PROTECT)
    included = models.BooleanField(default=True)        # True = feeds the estimate; Fa
    exclude_reason = models.TextField(blank=True, default="")

    class Meta:
        constraints = [models.UniqueConstraint(fields=["report", "comparable"], name="u
```

This is the recorded provenance (D-27): which comps were selected, which were excluded and why. A through-model (not a JSON blob of IDs) is deliberate — the audit trail needs real FK integrity, and `on_delete=PROTECT` **stops a referenced comp from silently vanishing from a historical report.**

3.6 `inventory.InventoryItem` — Phase 2 cost fields (additive migration)

Add (all nullable; no change to existing fields/behaviour):

```
    refurb_cost           = models.DecimalField(max_digits=12, decimal_places=2, null=T
```

```
    inbound_shipping_cost = models.DecimalField(max_digits=12, decimal_places=2, null=T
    est_outbound_shipping = models.DecimalField(max_digits=12, decimal_places=2, null=T
    est_packaging_cost    = models.DecimalField(max_digits=12, decimal_places=2, null=T
```

True cost = `(acquisition_cost or 0) + (refurb_cost or 0) + (inbound_shipping_cost or 0)`**. Per-sale** `est_outbound_shipping` **/** `est_packaging_cost` **fall back to the** `FeeSchedule` **defaults when null.**

3.7 Valuation strategy contract (`valuation/strategies.py`)

```
from dataclasses import dataclass

@dataclass
class EstimateResult:
    low: Decimal | None; median: Decimal | None; high: Decimal | None
    suggested: Decimal | None; fast_sale: Decimal | None
    patient: Decimal | None; min_acceptable: Decimal | None

class ValuationStrategy(Protocol):
    key: str
    def estimate(self, *, item, included_comps: list, inputs: dict) -> EstimateResult:

class CompBasedStrategy:          # key = "comp_based"
    # Uses the INCLUDED comps (prefer kind="sold"; fall back to active/auction if no so
    # NO advanced outlier math — the human curates inclusion (ValuationComparable.inclu
    #   low = min(prices); high = max(prices); median = statistics.median(prices)
    #   suggested = median;  patient = high;  fast_sale = low
    #   min_acceptable = left None for the human (or a cost-derived floor if desired) —
    ...

class CommodityManualStrategy:    # key = "commodity_manual"
    # inputs = {metal, weight_g, fineness (0-1), spot_price_per_g, buy_margin_pct}
    #   intrinsic = weight_g * fineness * spot_price_per_g          # spot is MANUALLY
    #   suggested = intrinsic;  patient = intrinsic
    #   fast_sale = intrinsic * (1 - buy_margin_pct/100)
    #   low/high/median = intrinsic (single point)
    ...

# "commodity_live" is registered but NOT implemented in Sprint 2 — it raises
# NotImplementedError("commodity_live: Sprint 3 — wire MetalsPriceAdapter").
```

All derived prices are starting heuristics; every one can be hand-edited on the report, and the manual override (with reason) is first-class (D-27).

3.8 `FeeCalculator` contract (`valuation/services.py`)

```
@dataclass
```

```
class ProfitBreakdown:
    sale_price: Decimal
    final_value_fee: Decimal; per_order_fee: Decimal; promoted_fee: Decimal; gst_on_fee
    outbound_shipping: Decimal; packaging: Decimal
    true_cost: Decimal
    total_deductions: Decimal; net_profit: Decimal; margin_pct: Decimal

def calculate_profit(*, sale_price, true_cost, schedule, outbound_shipping=None, packag
    # fvf       = (sale_price + outbound_shipping) * schedule.final_value_pct / 100
    # promoted  = sale_price * schedule.promoted_pct / 100
    # fees      = fvf + schedule.per_order_fee + promoted
    # gst       = fees * schedule.gst_pct / 100                        # AU GST on fees
    # ship/pack = args, else schedule.default_outbound_shipping / default_packaging_cos
    # net       = sale_price - (fees + gst + ship + pack + true_cost)
    # margin    = net / sale_price (guard /0)
```

Pure function. **No fee rate is hardcoded** — everything comes from the passed `schedule` (D-26).

3.9 Assisted research links (`research/links.py`)

```
def research_links(item) -> list[dict]:   # -> [{ "label": str, "url": str }]
    # q = item.title, else f'{attributes["brand"]} {attributes["model"]}'.strip(), else
    # Pure URL templating only — opens eBay's public website / Google in the browser. N
```

Links returned (AU domain): **eBay active** (`ebay.com.au/sch/i.html?_nkw=`), **eBay sold/completed** (`…&LH_Sold=1&LH_Complete=1`), **Terapeak** (deep-link to Seller Hub research where a query URL is practical, else the eBay sold search), **Google**, **Google Images** (`tbm=isch`), **exact-title** (quoted query), **"brand/model + sold"**. Catalogue-specific sources (Scott/SG/Renniks/Numista) are deferred (D-29).

---

4. Serializers (DRF)

- `ComparableSerializer` — straightforward; `item` writable on create.
- `ResearchRecordSerializer` — straightforward; `links` validated as a list of `{label,url}` .
- `FeeScheduleSerializer` — **read-only via the API** in Sprint 2 (managed in admin).
- `ValuationComparableSerializer` — `comparable` (id), `included` , `exclude_reason` ; nested read includes the comp summary.
- `ValuationReportSerializer` :
  - Read: all fields + nested `comp_links` (provenance) + a computed `profit_projection` (see below).

- Write/create: accepts `strategy`, `inputs`, a list of comp selections (`[{comparable, included, exclude_reason}]`), optional manual price/override fields. On create it runs the selected strategy to populate the estimate + price points, then persists; if `is_overridden`, **`override_reason` is required** (validation error otherwise).
- `profit_projection`: computed via `FeeCalculator` over the item's true cost + the report's `fee_schedule` (or the active schedule) at `fast_sale_price`, `suggested_price`, `patient_price` → a list of `ProfitBreakdown`-shaped objects.

- The Phase-1 `InventoryItemDetailSerializer` is extended with: the new cost fields, `comps_count`, and a `current_valuation` summary (id, suggested/fast/patient prices, confidence).

---

5. API endpoints (Sprint 2 additions, all under `/api/`)

Method	Path	Purpose
GET / POST	`/api/comparables/`	List (filter `item`, `kind`; order `observed_on`, `price`) / create.
GET/PATCH/DELETE	`/api/comparables/{id}/`	Edit / delete a comp.
GET / POST	`/api/research-records/`	List (filter `item`) / create.
GET/PATCH/DELETE	`/api/research-records/{id}/`	Edit / delete.
GET	`/api/fee-schedules/`, `/api/fee-schedules/{id}/`	Read-only (admin-managed).
GET / POST	`/api/items/{id}/valuation-reports/`	List the item's reports / create one (runs the strategy).
GET/PATCH/DELETE	`/api/valuation-reports/{id}/`	Retrieve (incl. provenance + `profit_projection`) / edit / delete.
POST	`/api/valuation-reports/{id}/set-current/`	Mark controlling report; sync item `estimated_value/min_price/target_price`.
GET	`/api/items/{id}/research-links/`	Templated assisted-research URLs.
GET	`/api/valuation-reports/{id}/profit/?price=<n>`	`ProfitBreakdown` at an arbitrary manual price.

6. Frontend — the Research & Valuation cockpit

All additive, on the existing **Item Detail** screen (new collapsible sections); no new routes required. Stack unchanged (React + TS + Tailwind + React Query).

- **Comparables panel —** `ComparableList` + `ComparableForm` (kind, price, shipping, source, condition, url, observed_on, notes); edit/delete with `ConfirmDialog`.
- **Research log —** `ResearchLog`: dated notes + links; add-note form.
- **Assisted research —** `ResearchLinks`: a row of buttons (from `/research-links/`) opening each search in a new tab.
- **Valuation panel —** `ValuationPanel`: create/refresh a report — pick strategy (default from category: gold → `commodity_manual`, else `comp_based`); for `comp_based`, check/uncheck which comps are included (excluded ones take an `exclude_reason`); for `commodity_manual`, show weight / fineness / spot / margin inputs. Show computed low/median/high + the four price points; each price is editable; toggling **override** reveals a required reason. `ConfidenceControl` sets `confidence_score` + reason. "Make current" button → `set-current`.
- **Profit panel —** `ProfitBreakdown`: shows net profit + margin at fast-sale / suggested / patient (from `profit_projection`) and a manual-price input (calls `/profit/?price=`). Surfaces the true-cost inputs (`acquisition_cost`, `refurb_cost`, `inbound_shipping_cost`, packaging, outbound) as editable.
- **FeeSchedule**: managed in Django admin in Sprint 2 — **no dedicated settings UI** (kept thin).

New shared components: `ComparableForm`, `ComparableList`, `ResearchLog`, `ResearchLinks`, `ValuationPanel`, `ConfidenceControl`, `ProfitBreakdown`. New typed API modules: `comparables.ts`, `research.ts`, `valuation.ts`, `fees.ts`.

---

7. Acceptance criteria

Done when:

1. Migrations (incl. the additive `inventory` cost fields and the new `research` / `valuation` tables) **apply cleanly on real PostgreSQL in CI** — same gate as **D-22**. (No new GIN index is required in Sprint 2; `inputs` / `links` JSONB are stored, not queried.)
2. From the item detail screen you can add `Comparable`s of each `kind` and `ResearchRecord`s, and edit/delete them.

3. `FeeSchedule` is admin-editable and dated; **`FeeCalculator` computes net profit with zero hardcoded rates** (unit-tested), netting FVF + per-order + promoted + GST + shipping + packaging + true cost.
4. You can create a **`comp_based`** `ValuationReport`: select/exclude comps (exclusions carry a reason), get low/median/high + suggested/fast-sale/patient/min, set `confidence_score` + reason, override any price with a **required** reason, and **`set-current`** which syncs the item's `estimated_value/min_price/target_price`.
5. **`commodity_manual`** computes intrinsic value from manually-entered weight / fineness / spot (no live feed); `commodity_live` raises `NotImplementedError`.
6. **Expected profit** is shown at fast-sale / suggested / patient and at a manual price.
7. `/items/{id}/research-links/` returns correct templated URLs (eBay active, eBay sold, Terapeak, Google, Google Images, exact-title, brand/model+sold).
8. Exactly **one** **`is_current`** report per item (DB constraint enforced).
9. `manage.py backup` includes the new tables; restore on an empty DB still passes.
10. Backend + frontend test suites pass (§8); **PostgreSQL CI gate green** before Sprint 2 is closed.

---

8. Test plan

Backend (`pytest` + `pytest-django`):

- *Models:* one-current-per-item constraint rejects a second current report; `ValuationComparable` uniqueness; `Comparable` / `ResearchRecord` create.
- *FeeCalculator:* exact arithmetic over a fixture `FeeSchedule` (FVF on price+shipping, per-order, promoted, GST on fees, shipping/packaging fallback, margin); a zero-price guard.
- *Strategies:* `comp_based` median/low/high over a curated set, and that **excluded comps don't affect** the estimate; `commodity_manual` intrinsic formula; `commodity_live` raises.
- *Sync:* `set-current` clears the previous current and writes the item's denormalized fields.
- *API (`APIClient`):* comp + research CRUD; report create runs the strategy; `set-current`; `research-links` output; `profit` endpoint; override without a reason is rejected; unauthenticated rejected.
- *Backup:* new tables present after a backup→restore on an empty DB.

Frontend (Vitest + RTL — light): `ComparableForm` validation; `ValuationPanel` requires an `exclude_reason` when a comp is excluded and an `override_reason` when overriding;

`ProfitBreakdown` renders the three projections; API modules build correct requests.

Postgres CI gate (per D-22): the "Sprint 2 PostgreSQL Validation" workflow runs migrate + seed + tests on a Postgres service and must be green before closing Sprint 2.

---

9. Seed data (extend `apps/core/management/commands/seed.py`, idempotent)

- One **`FeeSchedule`** named e.g. "eBay AU (placeholder)" with **clearly-labelled placeholder rates** — a comment must state these are *not* verified eBay rates and must be set by Regan before relied upon.
- A handful of **`Comparable`**s (a few `sold`, an `active`, a `dealer`) on an existing sample item, with `observed_on` dates.
- One **`comp_based ValuationReport`** marked `is_current` for that item, with a couple of comps included and one excluded (with a reason).
- The **gold** sample item gets `commodity_manual inputs` (weight/fineness/spot) and a current report, to exercise that path.

---

10. Out of scope for Sprint 2 (restated — D-30)

Live `MetalsPriceAdapter` / any live metals pricing · eBay Browse API (or any eBay call) · OCR · AI identification · listing generation · `SaleRecord` / realized-profit dashboard · full per-category attribute schemas · automated sold-data ingestion · scraping · advanced outlier/statistical math. Each has a seam; filling it is a later sprint.

Hand-off notes for the implementing agent

- Build the models + migrations and get them green on Postgres **before** the UI — re-use the Django admin to enter a few comps and a fee schedule and sanity-check the `FeeCalculator` and `set-current` sync by hand.
- **Never hardcode a fee rate.** If you find yourself typing a percentage into code, it belongs on `FeeSchedule`.
- The human curates comp inclusion — **do not** add statistical outlier detection. The estimate is just median + range over the included set.
- Keep the `commodity_manual` path arithmetic-only on hand-entered inputs; **do not** call or implement `MetalsPriceAdapter`.

- Assisted research is URL templating only — **no fetching, no scraping, no API.**
- When ambiguous, prefer the smaller implementation that satisfies §7.